フロントエンドとの繋ぎこみ（第４章）

Solana Developer Bootcamp 2026 🇯🇵

自己紹介

- Name: asuma
- Role: Co-Founder / CTO @DaikoAI
- Links:
  - posaune0423.com
  - 𝕏: @0xasuma_jp
  - Github: @posaune0423
  - Linkedin: @posaune0423

講義内容

フロントエンドからブロックチェーンまでのデータの流れ

フロントエンド実装での注意点

よくある追加実装

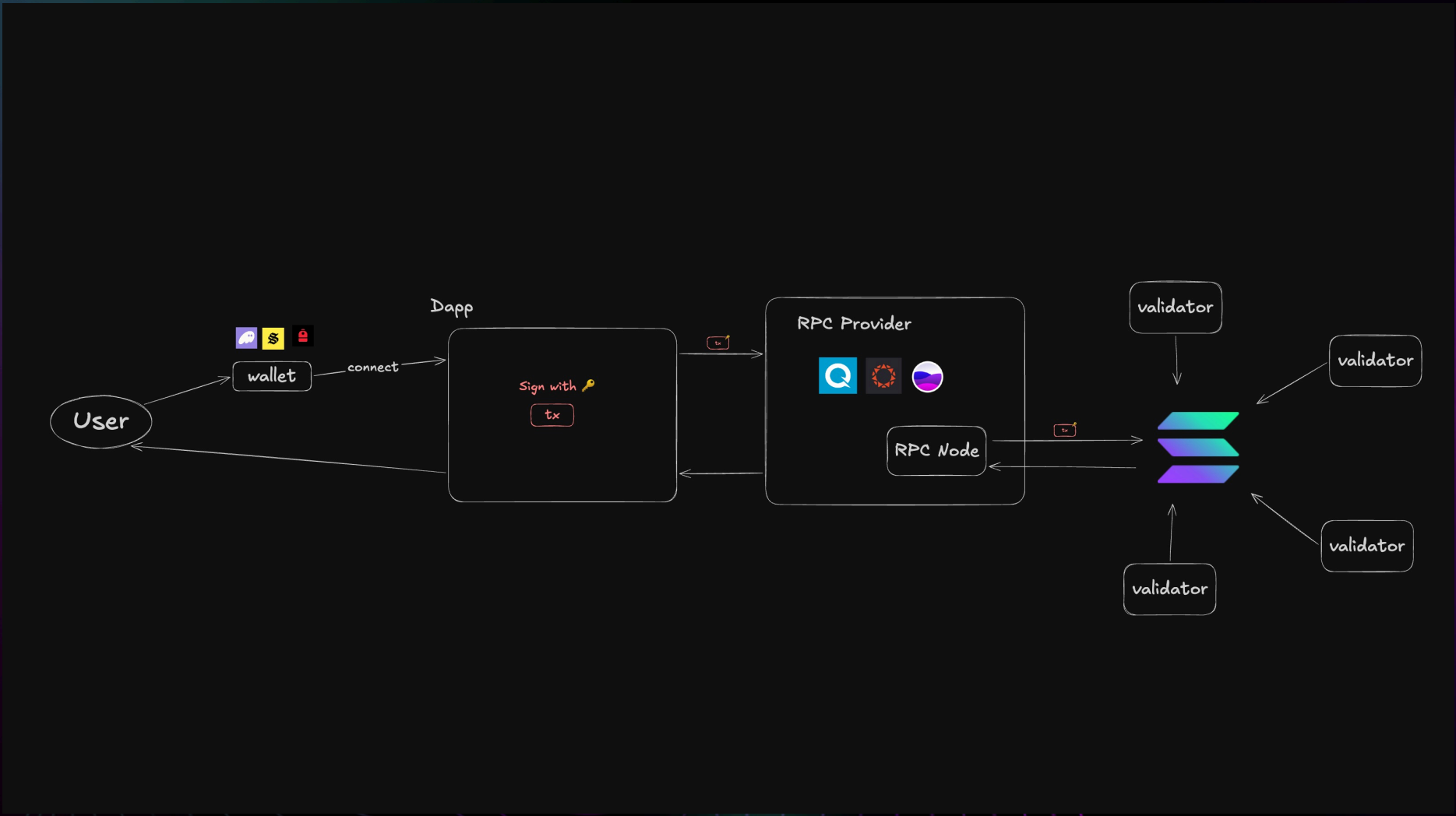
まとめ

フロントエンドからブロックチェーンまでのデータの流れ

1. Wallet Connect & sign tx
2. txの組み立て & RPCへ送信
3. RPC送信以降の流れ

フロントエンドからブロックチェーンまでのデータの流れ

全体像

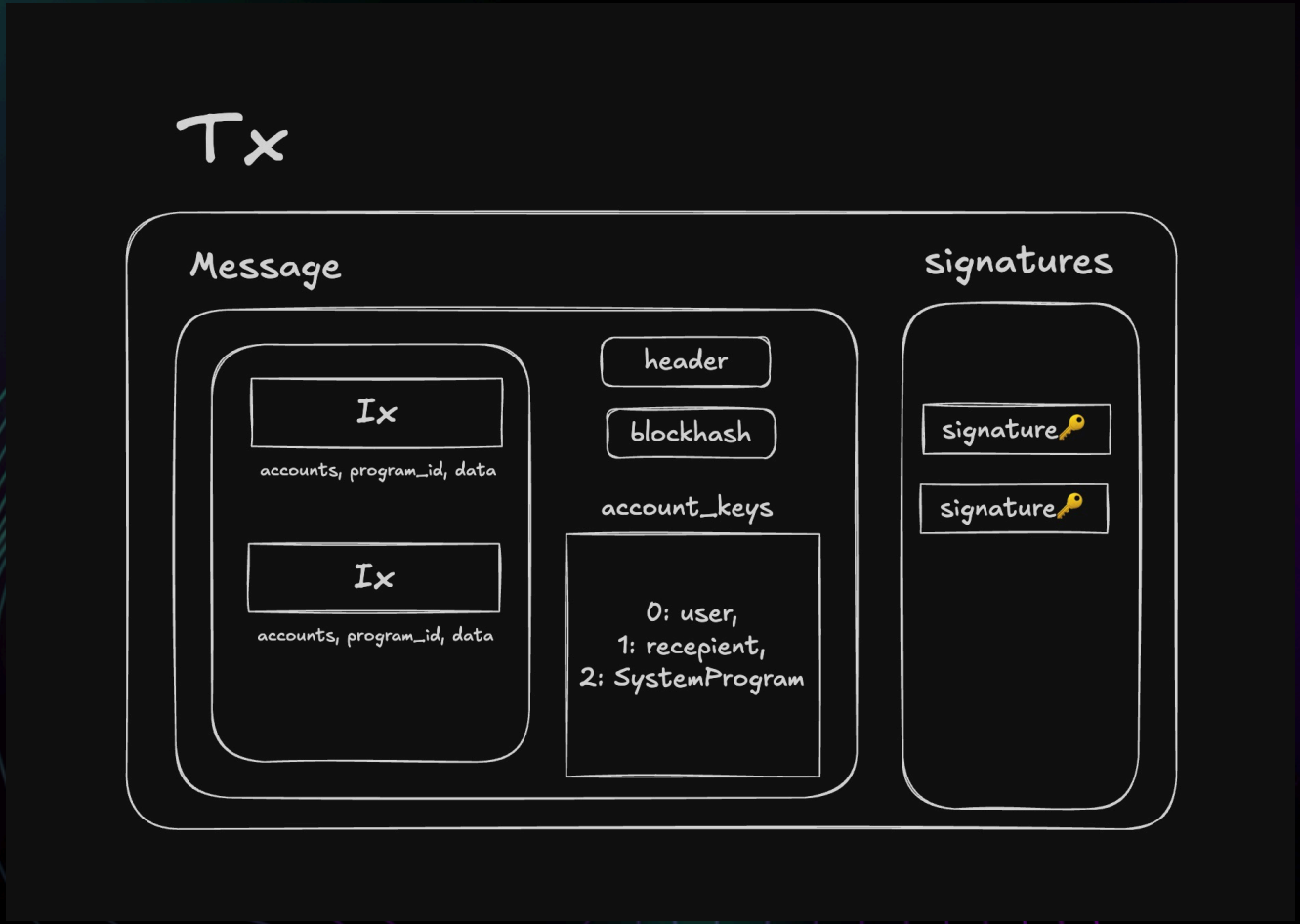


1. Wallet Connect & sign tx

```
function ConnectButton() {
  const { connectors, connect } = useWalletConnection()

  return (
    <div>
      {connectors.map((connector) => (
        <button key={connector.id} onClick={() => connect(connector.id)}>
          Connect {connector.name}
        </button>
      ))}
    </div>
  )
}
```

Solanaのtxの中身



2. txの組み立て & RPCへ送信

RPC(Provider)の役割:

BlockchainのNodeを運用しuserからの署名付きtxを受け取り
blockchainに送信するapiなどを提供しているインフラ事業者

HELIUS

QuickNode

alchemy

TRITON

2. txの組み立て & RPCへ送信

`@anchor-lang/core` は現状solana/kit未support. 最新の `@solana/kit` でanchor programを型安全に使うには、

```
codama run js
```

などでanchor programのidlからinstruction生成codeなどを自動生成する事で `@solana/kit` でも型安全にanchor programを呼び出すことができます

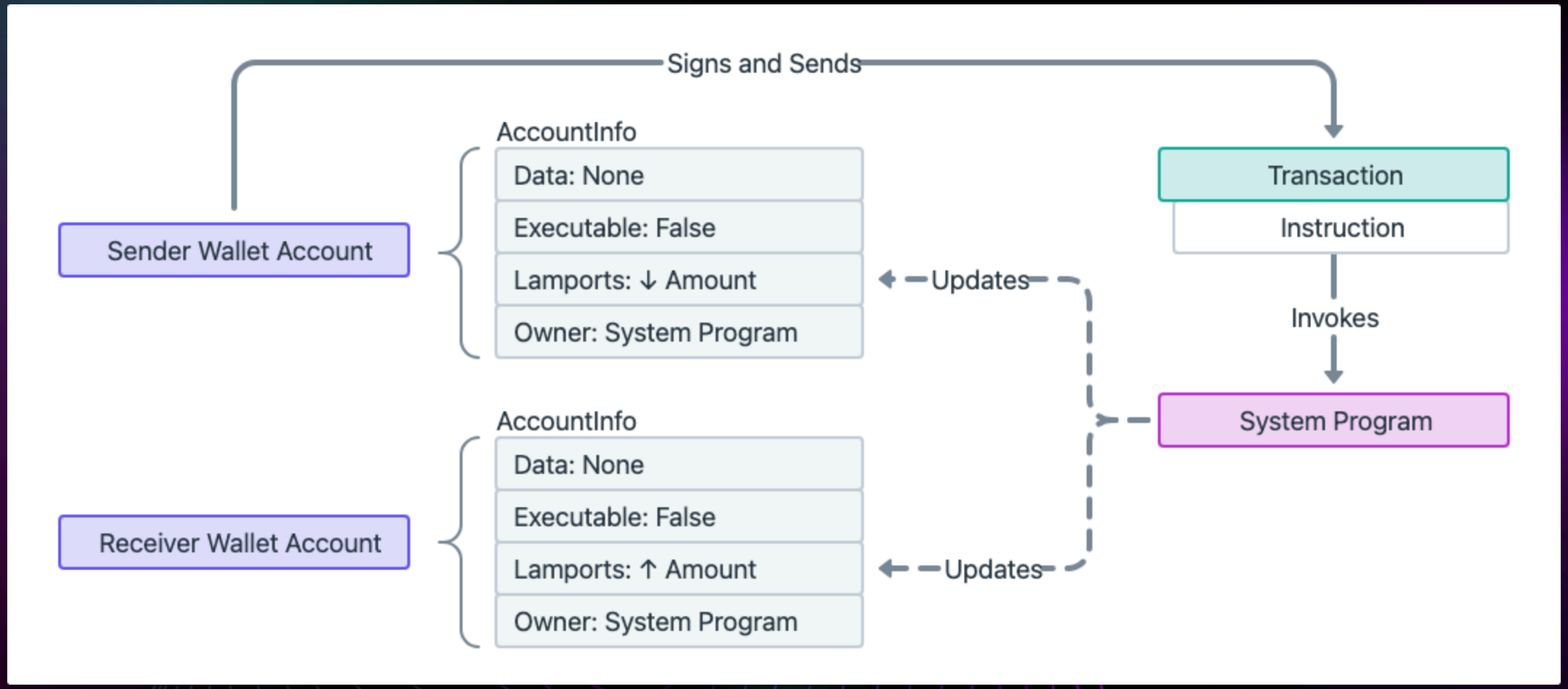
2. txの組み立て & RPCへ送信

codamaでgenerateした関数でinstructionを生成し `prepareAndSend()` で組み立てたtxをsubmitできます

```
import { useSendTransaction } from '@solana/react-hooks'

function SendPrepared({ instructions }) {
  const { prepareAndSend, isSending, status, signature, error } =
    useSendTransaction()
  return (
    <div>
      <button
        disabled={isSending}
        onClick={() ⇒ prepareAndSend({ instructions })}
      >
        {isSending ? 'Submitting…' : 'Send transaction'}
      </button>
      <p>Status: {status}</p>
      {signature ? <p>Signature: {signature}</p> : null}
      {error ? <p role="alert">{String(error)}</p> : null}
    </div>
  )
}
```

3. RPC送信以降の流れ



3. RPC送信以降の流れ

1. RPC Nodeがuserから受け取ったsigned txをleader scheduleに従ってcurrent leader nodeに投げる
2. Leader Nodeがtxを並列処理
3. program invokeによりaccountのstateが変化
    - solのtransferであれば単にlamports残高が変化
    - spl tokenのtransferであればdata account(ATA)のamountが変化
4. Leader Nodeがblock生成し他validatorにbroadcast

フロントエンド実装での注意点

1. 推奨libとlegacy lib
2. RPCの冗長設計
3. error handling
4. セキュリティ

1. 推奨libとlegacy lib

Latest ✅

`@solana/client`, `@solana/react-hooks`, `@solana/kit`

Legacy ❌

`@solana/web3.js`

※とはいえ、結構いろんなsdkやlibがまだlegacyなweb3.jsに依存している状況

2. RPCの冗長設計

RPCサービスも完璧ではありません。AWSやCloudflareなどと同じ様にサービスがダウンすることもあるのであらかじめ冗長構成を取っておくのが本番環境では重要です。

```
import { RpcTransport } from '@solana/rpc-spec'
import { RpcResponse } from '@solana/rpc-spec-types'
import { createHttpTransport } from '@solana/rpc-transport-http'

// Create a transport for each RPC server
const transports = [
  createHttpTransport({ url: 'https://mainnet-beta.my-server-1.com' }),
  createHttpTransport({ url: 'https://mainnet-beta.my-server-2.com' }),
  createHttpTransport({ url: 'https://mainnet-beta.my-server-3.com' }),
]

// Create a wrapper transport that distributes requests to them
let nextTransport = 0
async function roundRobinTransport<TResponse>(
  ...args: Parameters<RpcTransport>
): Promise<RpcResponse<TResponse>> {
  const transport = transports[nextTransport]
  nextTransport = (nextTransport + 1) % transports.length
  return await transport(...args)
}
```

3. error handling

rpc独自のエラーとdomain logicに関するエラーは分けて管理するべき。

基本的なrpc周りのerrorは `@solana/errors` というlibがまとめてくれている。

3. error handling

代表的なrpc error

Code	Message	Explanation
-32001	Block Cleaned Up	システムアップグレードに関連する問題の可能性があります。運用元にお問い合わせください。
-32002	Transaction simulation failed	多くの場合、無効な命令やパラメータ、あるいはblockhashが原因です。
-32003	Signature verification failure	署名が誤っている、もしくは不足している場合によく発生します。
-32004	Block not available for slot	一時的なエラーです。リトライを推奨します。
-32005	Node is unhealthy	ノードの遅延が原因です。ノードを変更するか、リトライしてください。
-32007	Slot skipped or missing	リクエストしたブロックが存在しません。Solana Explorerなどで確認してください。
-32009	Slot missing in long-term storage	履歴ブロックデータが利用できません。
-32010	Excluded from account indexes	無効なペイロード、または未サポートのRPCメソッドです。
-32013	Signature length mismatch	署名の形式が正しくありません。

3. error handling

```
import { SOLANA_ERROR__TRANSACTION__MISSING_SIGNATURE, SOLANA_ERROR__TRANSACTION__FEE_PAYER_SIGNATURE_MISSING, isSolanaError } from '@solana/errors'
import { assertIsFullySignedTransaction,getSignatureFromTransaction } from '@solana/transactions'

try {
  const transactionSignature = getSignatureFromTransaction(tx)
  assertIsFullySignedTransaction(tx)Ï
  /* ... */
} catch (e) {
  if (isSolanaError(e, SOLANA_ERROR__TRANSACTION__SIGNATURES_MISSING)) {
    displayError(
      "We can't send this transaction without signatures for these addresses:\n- %s",
      // The type of the `context` object is now refined to contain `addresses`.
      e.context.addresses.join('\n- '),
    )
    return
  } else if (
    isSolanaError(e, SOLANA_ERROR__TRANSACTION__FEE_PAYER_SIGNATURE_MISSING)
  ) {
    if (!tx.feePayer) {
      displayError('Choose a fee payer for this transaction before sending it')
    } else {
      displayError('The fee payer still needs to sign for this transaction')
    }
    return
  }
  throw e
}
```

4. セキュリティ

よくある追加実装

よくある追加実装

1.payerを使ったgasless tx

2. indexerを使用したデータの集計、読み取り

1. payerを使ったgasless tx

Solanaは複数署名者・複数権限者を前提にしたtransaction model.

`setTransactionMessageFeePayer(PLACEHOLDER_FEE_PAYER, tx)`などで`payer`を指定する事でsenderとpayerを分離してgas sponcerをnativeに実現可能

Alchemyなどのサービスを使うと以下の様に実装できます

```
const { value: bh } = await rpc.getLatestBlockhash().send()

const msg = pipe(
  createTransactionMessage({ version: 0 }),
  (tx) ⇒
    setTransactionMessageFeePayer(
      address('Amh6quo1FcmL16Qmzdugzjq3Lv1zXzTW7ktswyLDzits'), // placeholder
      tx,
    ),
  (tx) ⇒ setTransactionMessageLifetimeUsingBlockhash(bh, tx),
  (tx) ⇒
    appendTransactionMessageInstructions(
      [
        getTransferSolInstruction({
          source: user,
          destination: address(process.env.RECIPIENT!),
          amount: lamports(1_000_000n),
        }),
      ],
      tx,
    ),
)
```

payerを使ったgasless tx

```
const sponsored = await fetch(rpcUrl, {
  method: 'POST',
  headers: { 'content-type': 'application/json' },
  body: JSON.stringify({
    jsonrpc: '2.0',
    id: 1,
    method: 'alchemy_requestFeePayer',
    params: [
      {
        policyId: process.env.ALCHEMY_POLICY_ID!,
        serializedTransaction: getBase64EncodedWireTransaction(
          compileTransaction(msg),
        ),
      },
    ],
  }),
})
  .then((r) => r.json())
  .then((r) => r.result.serializedTransaction)

const tx = getTransactionDecoder().decode(getBase64Decoder().decode(sponsored))
const signed = await partiallySignTransaction([user], tx)

await rpc
  .sendTransaction(getBase64EncodedWireTransaction(signed), {
    encoding: 'base64',
  })
  .send()
```

2. indexerを使用したデータのクエリ・集計

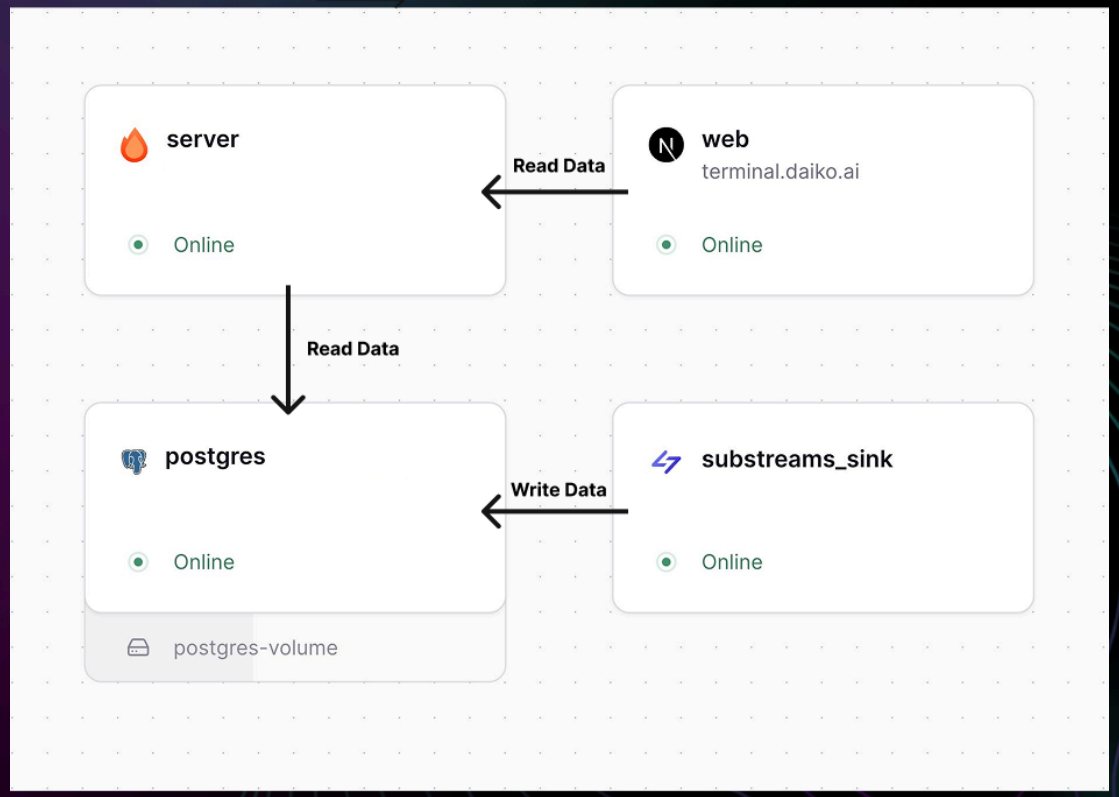
そもそもindexerとは？

オンチェーン上のデータをrealtimeにsubscribe / 整形してapplicationに合わせたread modelとして永続化するservice

どういう時にindexerが必要になるのか

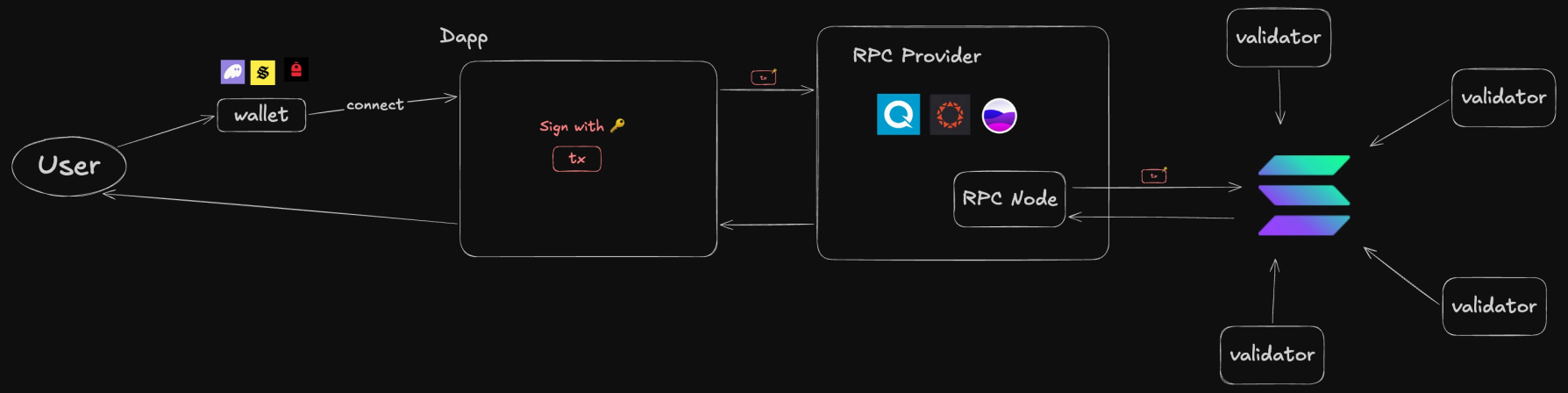
- filter, searchなどonchain dataに対してqueryしたり集計が必要な場合
- 複数プロトコルや複数コントラクトのデータをaggregateしたい時
- DEXのchartなどの時系列データの整形・表示

Substreamというサービスを使うとgraphQLやSQLでデータを簡単にfilter / queryできるようになる



まとめ

まとめ


User
wallet
connect
Dapp
Sign with
tx
RPC Provider
RPC Node
validator
validator
validator
validator

まとめ

フロントエンドからブロックチェーンまでのデータの流れ

1. Wallet Connect & sign tx
2. txの組み立て & RPCへ送信
3. RPC送信以降の流れ

フロントエンド実装での注意点

1. 推奨libとlegacy lib
2. RPCの冗長設計
3. error handling
4. セキュリティ

よくある追加実装

1. payerを使ったgasless tx
2. indexerを使用したデータのクエリ・集計

Thank you for listening

参考文献・サービス

- Substream
- Alchemy
- @solana/kit
- @solana/errors
- https://solana.com/docs/core/transactions/transaction-structure
- https://solana.com/docs/frontend
- https://www.youtube.com/watch?v=2T3DOMv7iR4